

Informe de proyecto: Sistema web de materiales

Alacena & Molde

Fecha de elaboración: 04/10/2026

Captura de código y estructura del proyecto

A continuación se muestran bloques de código reales del proyecto, presentados con un estilo visual tipo captura de editor de texto, para facilitar la explicación técnica del CRUD de materiales y la estructura del sistema.

1. Introducción

Este proyecto tuvo como objetivo desarrollar una aplicación web con Django para gestionar materiales de una repostería. Se implementó un CRUD funcional que permite visualizar, registrar, actualizar y eliminar materiales, manteniendo una estructura organizada y con validaciones necesarias para evitar errores en la entrada de datos.

2. Objetivos

- Listar materiales mediante consultas ORM de Django.
- Crear nuevos registros desde un formulario web.
- Editar materiales ya existentes.
- Eliminar materiales con confirmación.
- Mantener validación de campos, control de errores 404 y protección CSRF.

3. Requisitos funcionales

La aplicación debía incorporar CRUD completo, uso de `get_object_or_404()`, manejo adecuado de formularios, rutas específicas para cada operación y una vista de listado con acceso directo a cada acción. Además, se contempló validación para campos vacíos y valores inválidos.

4. Desarrollo del sistema

El módulo de materiales se estructura con una app Django llamada materiales. Este módulo incluye un modelo que almacena los datos, un formulario de validación, las vistas de lógica y una serie de templates para mostrar la interfaz. El patrón seguido fue el típico de Django MVC, aprovechando el ORM para gestionar todas las operaciones contra la base de datos.

4.1 Modelo de datos

materiales/models.py

```

materiales/models.py
from django.db import models

class Material(models.Model):
    nombre = models.CharField(max_length=100)
    categoria = models.CharField(max_length=50)
    stock = models.PositiveIntegerField()
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    descripcion = models.TextField(blank=True)

    def __str__(self):
        return self.nombre

```

4.2 Formularios y validación

materiales/forms.py

```

materiales/forms.py
from django import forms

from .models import Material

class MaterialForm(forms.ModelForm):
    class Meta:
        model = Material
        fields = ["nombre", "categoria", "stock", "precio", "descripcion"]
        widgets = {
            "nombre": forms.TextInput(attrs={"class": "form-control"}),
            "categoria": forms.TextInput(attrs={"class": "form-control"}),
            "stock": forms.NumberInput(attrs={"class": "form-control"}),
            "precio": forms.NumberInput(attrs={"class": "form-control", "step": "0.01"}),
            "descripcion": forms.Textarea(attrs={"class": "form-control", "rows": 4}),
        }

    def clean_precio(self):
        precio = self.cleaned_data.get("precio")
        if precio is not None and precio <= 0:
            raise forms.ValidationError("El precio debe ser mayor que cero.")
        return precio

    def clean_stock(self):
        stock = self.cleaned_data.get("stock")
        if stock is not None and stock < 0:
            raise forms.ValidationError("El stock no puede ser negativo.")
        return stock

```

4.3 Vistas del CRUD

materiales/views.py

```

materiales/views.py
from django.shortcuts import get_object_or_404, redirect, render

from .forms import MaterialForm
from .models import Material

def listar_materiales(request):
    materiales = Material.objects.all().order_by("nombre")
    return render(request, "materiales/lista.html", {"materiales": materiales})

def crear_material(request):
    if request.method == "POST":
        form = MaterialForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("materiales:lista")
    else:
        form = MaterialForm()

    return render(request, "materiales/formulario.html", {"form": form, "accion": "Crear"})

def editar_material(request, pk):
    material = get_object_or_404(Material, pk=pk)

    if request.method == "POST":
        form = MaterialForm(request.POST, instance=material)

```

4.4 Rutas del sistema

materiales/urls.py

```
materiales/urls.py
from django.urls import path

from . import views

app_name = "materiales"

urlpatterns = [
    path("", views.listar_materiales, name="lista"),
    path("crear/", views.crear_material, name="crear"),
    path("editar/<int:pk>/", views.editar_material, name="editar"),
    path("eliminar/<int:pk>/", views.eliminar_material, name="eliminar"),
]
```

4.5 Templates

Se crearon las plantillas para listar, crear, editar y confirmar eliminación.

materiales/templates/materiales/lista.html

```
materiales/templates/materiales/lista.html
{% extends 'catalogo/base.html' %}

{% block title %}Materiales | Alacena & Molde{% endblock %}

{% block content %}
<main class="materials-page">
  <section class="materials-header">
    <div>
      <p class="eyebrow">MATERIALES</p>
      <h1>Inventario de <em>materiales</em></h1>
    </div>
    <a class="primary-button" href="{% url 'materiales:crear' %}">Agregar material <span>⌕</span></a>
  </section>

  {% if materiales %}
  <div class="materials-table-wrap">
    <table class="materials-table">
      <thead>
        <tr>
          <th>Nombre</th>
          <th>Categoría</th>
          <th>Stock</th>
          <th>Precio</th>
          <th>Descripción</th>
          <th>Acciones</th>
        </tr>
      </thead>
      <tbody>

```

materiales/templates/materiales/formulario.html

```
materiales/templates/materiales/formulario.html
{% extends 'catalogo/base.html' %}

{% block title %}{% accion %} material | Alacena & Molde{% endblock %}

{% block content %}
<main class="form-page">
  <section class="form-card">
    <p class="eyebrow">{% accion|upper %} MATERIAL</p>
    <h1>{% accion %} material</h1>

    <form method="post" novalidate>
      {% csrf_token %}
      {% form.non_field_errors %}

      {% for field in form %}
      <div class="field-group">
        <label for="{% field.id_for_label %}">{% field.label %}</label>
        {{ field }}
        {% if field.errors %}
        <div class="error-text">{% field.errors|striptags %}</div>
        {% endif %}
      </div>
      {% endfor %}

      <div class="form-actions">
        <button class="primary-button" type="submit">Guardar</button>
        <a class="secondary-button" href="{% url 'materiales:lista' %}">Cancelar</a>
      </div>

```

materiales/templates/materiales/eliminar.html

```

materiales/templates/materiales/eliminar.html
{% extends 'catalogo/base.html' %}

{% block title %}Eliminar material | Alacena & Molde{% endblock %}

{% block content %}
<main class="form-page">
  <section class="form-card confirm-card">
    <p class="eyebrow">CONFIRMACIÓN</p>
    <h1>Eliminar material</h1>
    <p>¿Estás seguro de que deseas eliminar <strong>{{ material.nombre }}</strong>?</p>

    <form method="post">
      {% csrf_token %}
      <div class="form-actions">
        <button class="primary-button danger-button" type="submit">Confirmar eliminación</button>
        <a class="secondary-button" href="{% url 'materiales:lista' %}">Cancelar</a>
      </div>
    </form>
  </section>
</main>
{% endblock %}

```

5. Pruebas realizadas

Se ejecutaron pruebas para comprobar el correcto funcionamiento del sistema. Se validaron los casos de listado, creación, edición, cancelación de eliminación, confirmación, manejo de 404, validación de errores y seguridad CSRF.

materiales/tests.py

```

Verificación
from django.test import Client, TestCase
from django.urls import reverse

from .models import Material

class MaterialCrudTests(TestCase):
    def setUp(self):
        self.material = Material.objects.create(
            nombre="Harina 0000",
            categoria="Ingredientes",
            stock=25,
            precio=150.00,
            descripcion="Harina para pan y repostería.",
        )
        self.list_url = reverse("materiales:lista")
        self.crear_url = reverse("materiales:crear")
        self.editar_url = reverse("materiales:editar", args=[self.material.pk])
        self.eliminar_url = reverse("materiales:eliminar", args=[self.material.pk])

    def test_listado_muestra_materiales(self):
        response = self.client.get(self.list_url)
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Harina 0000")
        self.assertTemplateUsed(response, "materiales/lista.html")

    def test_creacion_valida(self):
        payload = {

```

6. Resultados verificados

Verificación

Verificación

```
from django.test import Client, TestCase
from django.urls import reverse

from .models import Material

class MaterialCrudTests(TestCase):
    def setUp(self):
        self.material = Material.objects.create(
            nombre="Harina 0000",
            categoria="Ingredientes",
            stock=25,
            precio=150.00,
            descripcion="Harina para pan y repostería.",
        )
        self.list_url = reverse("materiales:lista")
        self.crear_url = reverse("materiales:crear")
        self.editar_url = reverse("materiales:editar", args=[self.material.pk])
        self.eliminar_url = reverse("materiales:eliminar", args=[self.material.pk])

    def test_listado_muestra_materiales(self):
        response = self.client.get(self.list_url)
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Harina 0000")
        self.assertTemplateUsed(response, "materiales/lista.html")

    def test_creacion_valida(self):
        payload = {
```

7. Conclusión

El proyecto cumple con los requisitos solicitados para un CRUD web de materiales en Django. La aplicación se encuentra operativa, con enlaces funcionales, validaciones, manejo de errores y seguridad básica mediante CSRF. Además, la estructura es escalable y preparada para continuar con nuevas funcionalidades del catálogo de repostería.